

translated by Google

Google utiliza tecnología de IA para traducir contenido a tu idioma preferido. Las traducciones realizadas con IA pueden contener errores.

[Switch to English](#)

Cómo guardar datos en una base de datos local usando Room

Parte de **Android Jetpack** (<https://developer.android.com/jetpack?hl=es-419>).

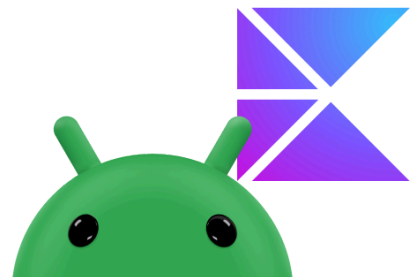


Probar con Kotlin Multiplatform

Kotlin Multiplatform permite compartir la capa de la base de datos con otras plataformas. Aprende a configurar y trabajar con la base de datos de Room en KMP

[Configurar la base de datos de Room para KMP →](#)

(<https://developer.android.com/kotlin/multiplatform/room?hl=es-419>)



Las apps que controlan grandes cantidades de datos estructurados pueden beneficiarse con la posibilidad de conservar esos datos localmente. El caso de uso más común consiste en almacenar en caché datos relevantes para que el dispositivo no pueda acceder a la red, de modo que el usuario pueda explorar ese contenido mientras está sin conexión.

La biblioteca de persistencias Room brinda una capa de abstracción para SQLite que permite acceder a la base de datos sin problemas y, al mismo tiempo, aprovechar toda la tecnología de SQLite. En particular, Room brinda los siguientes beneficios:

- Verificación del tiempo de compilación de las consultas en SQL
- Anotaciones de conveniencia que minimizan el código estándar repetitivo y propenso a errores

- Rutas de migración de bases de datos optimizadas

Recomendamos utilizar Room en lugar de usar las APIs de SQLite directamente

(<https://developer.android.com/training/data-storage/sqlite?hl=es-419>).

Configuración

Para usar Room en tu app, agrega las siguientes dependencias al archivo `build.gradle.kts` del módulo. Room 3.0 requiere KSP para el procesamiento de anotaciones.

<u>Kotlin</u> <u>build.gradle.kts</u>	<u>Groovy</u> <u>build.gradle</u>
<pre>dependencies { val room_version = "3.0.3" implementation("androidx.room3:room3-runtime:\$room_version") ksp("androidx.room3:room3-compiler:\$room_version") }</pre>	

Componentes principales

Estos son los tres componentes principales de Room:

- La clase de la base de datos
(<https://developer.android.com/reference/kotlin/androidx/room3/Database?hl=es-419>) que contiene la base de datos y sirve como punto de acceso principal para la conexión subyacente a los datos persistentes de la app
- Las entidades de datos
(<https://developer.android.com/training/data-storage/room/defining-data?hl=es-419>) que representan tablas de la base de datos de tu app
- Los objetos de acceso a datos (DAOs)
(<https://developer.android.com/training/data-storage/room/accessing-data?hl=es-419>) que proporcionan funciones que tu app puede usar para consultar, actualizar, insertar y borrar datos en la base de datos

La clase de base de datos proporciona a tu app instancias de los DAOs asociados con esa base de datos. A su vez, la app puede usar los DAOs para recuperar datos de la base de datos como instancias de objetos de entidad de datos asociados. La app también puede usar las entidades de datos definidas para actualizar filas de las tablas correspondientes o crear filas

nuevas para su inserción. En la figura 1, se muestran las relaciones entre los diferentes componentes de Room.

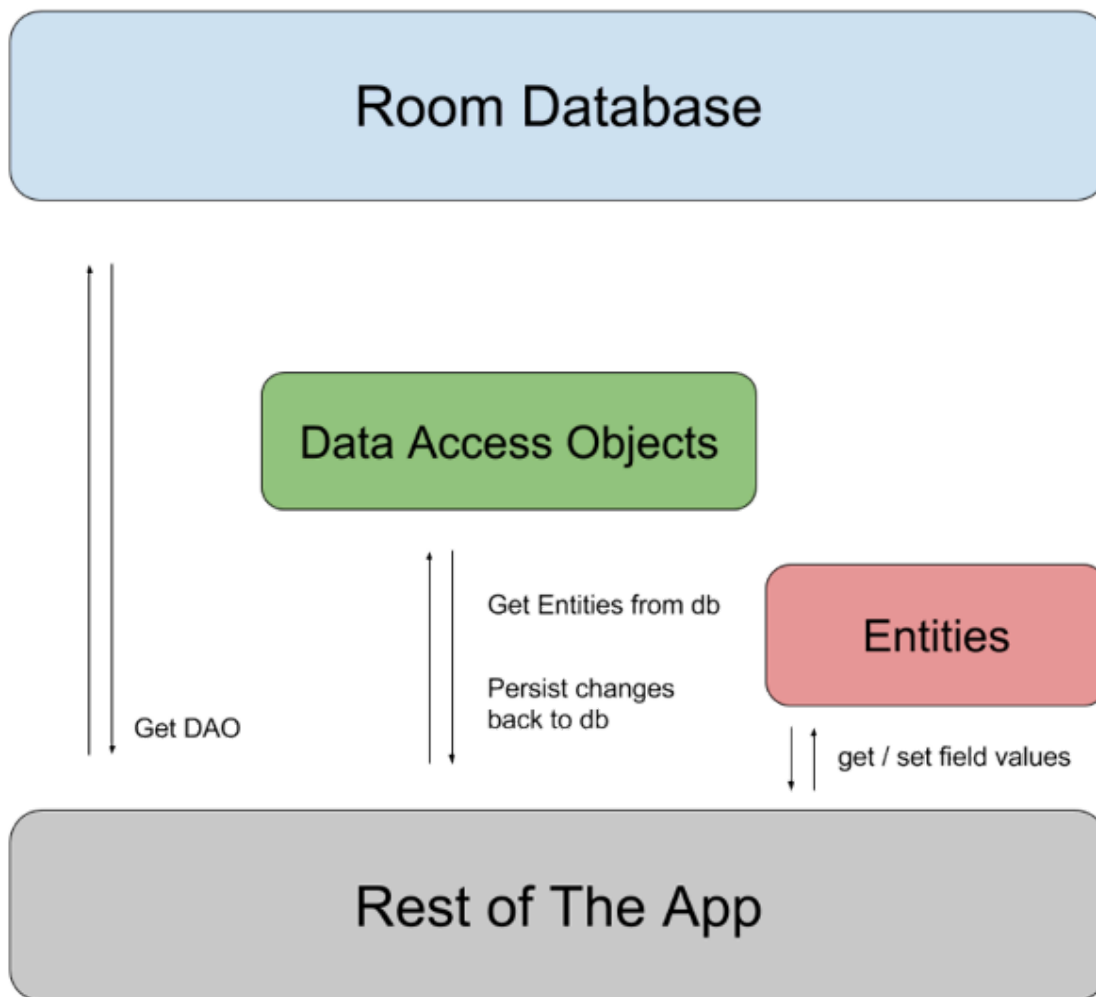


Figura 1. Diagrama de la arquitectura de la biblioteca de Room

Ejemplo de implementación

En esta sección, se presenta un ejemplo de implementación de una base de datos de Room con una sola entidad de datos y un DAO único.

Entidad de datos

El siguiente código define una entidad de datos `User`. Cada instancia de `User` representa una fila en una tabla `user` de en la base de datos de la app.

```
@Entity
data class User(
    @PrimaryKey val uid: Int,
    @ColumnInfo(name = "first_name") val firstName: String,
```

```

        @ColumnInfo(name = "last_name") val lastName: String
    )
}

```

ed612226d7d27b9d9aba8/room/src/main/java/com/example/snippets/room/index/IndexSnippets.kt#L33-L38)

Para obtener más información sobre las entidades de datos de Room, consulta [Cómo definir datos con entidades de Room](https://developer.android.com/training/data-storage/room/defining-data?hl=es-419)

(<https://developer.android.com/training/data-storage/room/defining-data?hl=es-419>).

Objeto de acceso a datos (DAO)

El siguiente código define un DAO llamado `UserDao`. `UserDao` proporciona los métodos que el resto de la app usa para interactuar con los datos de la tabla `user`.

```

@Dao
interface UserDao {
    @Query("SELECT * FROM user")
    suspend fun getAll(): List<User>

    @Query("SELECT * FROM user WHERE uid IN (:userIds)")
    suspend fun loadAllByIds(userIds: IntArray): List<User>

    @Query(
        """
        SELECT * FROM user
        WHERE first_name LIKE :first AND last_name LIKE :last LIMIT 1
        """
    )
    suspend fun findByName(first: String, last: String): User

    @Insert
    suspend fun insertAll(vararg users: User)

    @Delete
    suspend fun delete(user: User)
}

```

ed612226d7d27b9d9aba8/room/src/main/java/com/example/snippets/room/index/IndexSnippets.kt#L42-L63)

Para obtener más información sobre los DAOs, consulta [Cómo acceder a los datos con DAO de Room](https://developer.android.com/training/data-storage/room/accessing-data?hl=es-419) (<https://developer.android.com/training/data-storage/room/accessing-data?hl=es-419>).

Base de datos

Con el siguiente código, se define una clase `AppDatabase` para contener la base de datos. `AppDatabase` define la configuración de la base de datos y sirve como el punto de acceso principal de la app a los datos persistentes. La clase de la base de datos debe cumplir con las siguientes condiciones:

- La clase debe tener una anotación `@Database` (<https://developer.android.com/reference/kotlin/androidx/room3/Database?hl=es-419>) que incluya un array `entities` ([https://developer.android.com/reference/kotlin/androidx/room3/Database?hl=es-419#entities\(\)](https://developer.android.com/reference/kotlin/androidx/room3/Database?hl=es-419#entities())) que enumere todas las entidades de datos asociados con la base de datos.
- Debe ser una clase abstracta que extienda `RoomDatabase` (<https://developer.android.com/reference/kotlin/androidx/room3/RoomDatabase?hl=es-419>).
- Para cada clase DAO asociada con la base de datos, esta base de datos debe definir una función abstracta que no tome argumentos y muestre una instancia de la clase DAO.

```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
ed612226d7d27b9d9aba8/room/src/main/java/com/example/snippets/room/index/IndexSnippets.kt#L67-L70)
```

Nota: Si tu app se ejecuta en un solo proceso, debes seguir el patrón de diseño singleton cuando crees una instancia de un `AppDatabase` objeto. Cada instancia `RoomDatabase` es bastante costosa y rara vez necesitas acceder a varias instancias en un mismo proceso.

Si tu app se ejecuta en varios procesos, incluye `enableMultiInstanceInvalidation()` en tu invocación del creador de bases de datos. De esa manera, cuando tienes una instancia de `AppDatabase` en cada proceso, puedes invalidar el archivo de base de datos compartido en un proceso, y esta invalidación se propaga automáticamente a las instancias de `AppDatabase` dentro de otros procesos.

Uso

Después de definir la entidad de datos, el DAO y el objeto de base de datos, puedes usar el siguiente código para crear una instancia de la base de datos:

```
val db =
    Room.databaseBuilder<AppDatabase>(applicationContext, "database-name")
```

```
.setDriver(AndroidSQLiteDriver())
    .build()
ed612226d7d27b9d9aba8/room/src/main/java/com/example/snippets/room/index/IndexSnippets.kt#L76-L79)
```

Luego, puedes usar las funciones abstractas de `AppDatabase` para obtener una instancia del DAO. A su vez, puedes usar las funciones de la instancia del DAO para interactuar con la base de datos:

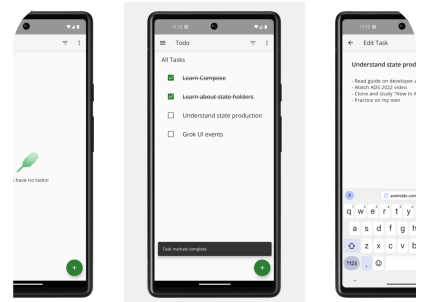
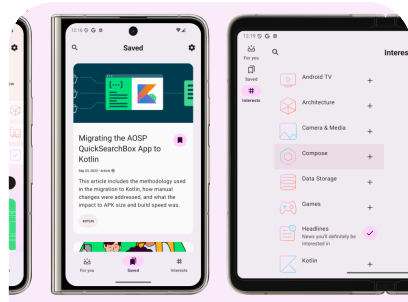
```
val userDao = db.userDao()
val users: List<User> = userDao.getAll()
ed612226d7d27b9d9aba8/room/src/main/java/com/example/snippets/room/index/IndexSnippets.kt#L86-L87)
```

Recursos adicionales

Si deseas obtener más información sobre Room, consulta los siguientes recursos adicionales:

Ejemplos

<https://github.com/android/compose-samples/tree/main/Jetcaster>



GitHub

Jetcaster sample



(<https://github.com/android/compose-samples/tree/main/Jetcaster>)

GitHub

Now in Android App

(<https://github.com/android/nowinandroid/tree/main>)

Learn how this app was designed and built in the design case study, architecture learning journe...

GitHub

Architecture

(<https://github.com/android/architecture-samples/tree/main>)

These samples showcase different architectural approaches to developing Android apps. In its differen...

El contenido y las muestras de código que aparecen en esta página están sujetas a las licencias que se describen en la [Licencia de Contenido \(https://developer.android.com/license?hl=es-419\)](https://developer.android.com/license?hl=es-419). Java y OpenJDK son marcas registradas de Oracle o sus afiliados.

Última actualización: 2026-09-17 (UTC)